

Подготовка к защите

Тема 1. Вариант 4

Борисюк Е. Г.

1. Короткий рассказ о работе

Я решал задачу линейного программирования двухэтапным симплекс-методом. Привёл ограничения к равенствам, добавил искусственные переменные для начального допустимого базиса, решил вспомогательную задачу. После достижения нулевой вспомогательной цели удалил искусственные столбцы, вернул исходную цель и продолжил симплекс-шаги. Получил $x^* = (0, 0, 0, 5)$, $z_{\min} = 10$. Этот ответ совпал с Excel Solver и собственной программой на Python.

2. Правильные условия и обозначения

$$z = 3x_1 + x_2 + 4x_3 + 2x_4 \rightarrow \min, \quad \begin{cases} 2x_1 + x_2 + x_3 \leq 8, \\ x_1 + x_3 + x_4 = 5, \\ x_2 + x_4 \geq 4, \\ x_i \geq 0. \end{cases}$$

Все исходные переменные неотрицательны. s_1 — запас первого ограничения, s_3 — избыток третьего. Индекс 3 выбран по номеру ограничения. Во втором ограничении равенство: запас или избыток добавлять нельзя.

$$2x_1 + x_2 + x_3 + s_1 = 8, \quad x_1 + x_3 + x_4 = 5, \quad x_2 + x_4 - s_3 = 4.$$

Для начального базиса добавляю a_2 во вторую строку и a_3 в третью.

$$w = a_2 + a_3 \rightarrow \min, \quad \begin{cases} 2x_1 + x_2 + x_3 + s_1 = 8, \\ x_1 + x_3 + x_4 + a_2 = 5, \\ x_2 + x_4 - s_3 + a_3 = 4. \end{cases}$$

Все переменные неотрицательны. Начальный базис: $s_1 = 8$, $a_2 = 5$, $a_3 = 4$, остальные переменные равны нулю.

Начальный план допустим для вспомогательной задачи; для исходной задачи он ещё не допустим, поскольку искусственные переменные ненулевые.

3. Как я заполнял и пересчитывал таблицы

Столбцы идут в порядке $x_1, x_2, x_3, x_4, s_1, s_3, a_2, a_3$. В строки переношу коэффициенты из равенств; справа записываю 8, 5, 4. Слева указываю базис s_1, a_2, a_3 . Их столбцы образуют единичную матрицу. Небазисные переменные равны нулю, значения базисных равны правым частям.

Во вспомогательной цели $c = (0, 0, 0, 0, 0, 0, 1, 1)$, $c_B = (0, 1, 1)$. Считаю каждую оценку $r_j = c_j - \sum_i c_{B_i} t_{ij}$. Например, $r_{x_1} = 0 - (0 \cdot 2 + 1 \cdot 1 + 1 \cdot 0) = -1$, $r_{x_4} = 0 - (0 \cdot 0 + 1 \cdot 1 + 1 \cdot 1) = -2$. Справа в нижней строке стоит значение цели $w = 0 \cdot 8 + 1 \cdot 5 + 1 \cdot 4 = 9$, а не оценка.

При отрицательной оценке выбираю первый подходящий столбец по индексу. Это правило Бланда; наиболее отрицательную оценку выбирать не обязательно. Затем считаю отношения b_i/t_{ik} только для $t_{ik} > 0$. Минимальное отношение определяет выходящую переменную и сохраняет неотрицательность правых частей. Нулевое отношение допускается.

Разрешающую строку делю на разрешающий элемент p : $R'_q = R_q/p$. Для остальных строк выполняю $R'_i = R_i - t_{ik}R'_q$. Пересчитываю всю строку, включая правую часть. Оценки вычисляю заново с коэффициентами нового базиса.

Пример первого пересчёта: $p = 2$, $R'_1 = R_1/2$, $R'_2 = R_2 - R'_1$, третья строка не меняется. Вторая правая часть становится $5 - 4 = 1$; коэффициент при x_2 становится $0 - 1/2 = -1/2$. После этого $c_B = (0, 1, 1)$, $w = 1 + 4 = 5$, $r_{x_2} = 0 - (0 \cdot 1/2 + 1 \cdot (-1/2) + 1 \cdot 1) = -1/2$.

4. Что я делал на каждом шаге ручного решения

Таблица I₀

В первой таблице выбираю x_1 , хотя у x_4 оценка ниже. Отношения 4 и 5; третья строка не участвует, поскольку коэффициент нулевой.

$$\theta_1 = 8/(2) = 4, \quad \theta_2 = 5/(1) = 5.$$

x_1 входит, s_1 выходит. Разрешающий элемент 2.

$$R'_1 = R_1/(2), \quad R'_2 = R_2 - (1)R'_1.$$

Таблица I₁

Выбираю x_2 . Отношения 8 и 4 для первой и третьей строк; во второй коэффициент отрицательный. Выходит a_3 . Вспомогательная цель уменьшается с 5 до 3.

$$\theta_1 = 4/(\frac{1}{2}) = 8, \quad \theta_3 = 4/(1) = 4.$$

x_2 входит, a_3 выходит. Разрешающий элемент 1.

$$R'_3 = R_3/(1), \quad R'_1 = R_1 - (\frac{1}{2})R'_3, \quad R'_2 = R_2 - (-\frac{1}{2})R'_3.$$

Таблица I₂

Выбираю x_3 . Отношения 4 и 6, поэтому выходит x_1 . Искусственная a_2 ещё остаётся в базисе. Цель уменьшается с 3 до 1.

$$\theta_1 = 2/(\frac{1}{2}) = 4, \quad \theta_2 = 3/(\frac{1}{2}) = 6.$$

x_3 входит, x_1 выходит. Разрешающий элемент $\frac{1}{2}$.

$$R'_1 = R_1/(\frac{1}{2}), \quad R'_2 = R_2 - (\frac{1}{2})R'_1.$$

Таблица I₃

Выбираю x_4 . Отношения $1/2$ и 4 для второй и третьей строк. Выходит a_2 ; цель становится нулевой.

$$\theta_2 = 1/(2) = \frac{1}{2}, \quad \theta_3 = 4/(1) = 4.$$

x_4 входит, a_2 выходит. Разрешающий элемент 2 .

$$R'_2 = R_2/(2), \quad R'_1 = R_1 - (-1)R'_2, \quad R'_3 = R_3 - (1)R'_2.$$

Таблица I₄

Теперь $a_2 = a_3 = 0$, базис x_3, x_4, x_2 . Получен допустимый исходный план $(0, 7/2, 9/2, 1/2)$. Удаляю искусственные столбцы. Нулевая вспомогательная цель означает допустимость, но ещё не оптимальность исходной цели.

Таблица II₀

Для основной цели $c_B = (4, 2, 1)$; $z = 4(9/2) + 2(1/2) + 7/2 = 45/2$. Например, $r_{x_1} = 3 - [4(3/2) + 2(-1/2) + 1(1/2)] = -5/2$. Выбираю x_1 , отношения 3 и 7 . Выходит x_3 , цель становится 15 .

$$\theta_1 = \frac{9}{2}/(\frac{3}{2}) = 3, \quad \theta_3 = \frac{7}{2}/(\frac{1}{2}) = 7.$$

x_1 входит, x_3 выходит. Разрешающий элемент $\frac{3}{2}$.

$$R'_1 = R_1/(\frac{3}{2}), \quad R'_2 = R_2 - (-\frac{1}{2})R'_1, \quad R'_3 = R_3 - (\frac{1}{2})R'_1.$$

Таблица II₁

Выбираю s_1 , оценка $-2/3$. Отношения 9 и 6 ; выходит x_2 . Цель уменьшается с 15 до 11 . Дополнительные переменные также могут входить в базис.

$$\theta_1 = 3/(\frac{1}{3}) = 9, \quad \theta_3 = 2/(\frac{1}{3}) = 6.$$

s_1 входит, x_2 выходит. Разрешающий элемент $\frac{1}{3}$.

$$R'_3 = R_3/(\frac{1}{3}), \quad R'_1 = R_1 - (\frac{1}{3})R'_3, \quad R'_2 = R_2 - (-\frac{1}{3})R'_3.$$

Таблица II₂

Выбираю s_3 , оценка -1 . Единственное допустимое отношение равно 1 , поэтому выходит x_1 . Цель становится 10 .

$$\theta_1 = 1/(1) = 1.$$

s_3 входит, x_1 выходит. Разрешающий элемент 1 .

$$R'_1 = R_1/(1), \quad R'_2 = R_2 - (-1)R'_1, \quad R'_3 = R_3 - (-2)R'_1.$$

Таблица II₃

Все оценки $(1, 1, 2, 0, 0, 0)$ неотрицательны. Считаю базис: $s_3 = 1, x_4 = 5, s_1 = 8$. Остальные переменные нулевые. Запас первого ограничения равен 8, избыток третьего равен 1.

5. Ответ и независимая проверка

$$x^* = (0, 0, 0, 5), \quad z_{\min} = 10.$$

Подстановка: $2 \cdot 0 + 0 + 0 = 0 \leq 8$; $0 + 0 + 5 = 5$; $0 + 5 = 5 \geq 4$. Из равенства $x_4 = 5 - x_1 - x_3$, поэтому

$$z = 3x_1 + x_2 + 4x_3 + 2(5 - x_1 - x_3) = 10 + x_1 + x_2 + 2x_3 \geq 10.$$

При $x_1 = x_2 = x_3 = 0$ достигается нижняя граница. Оптимум единственный: для равенства $z = 10$ все три неотрицательных слагаемых должны равняться нулю; тогда $x_4 = 5$.

6. Как я сделал решение в Excel

В диапазон B5:E5 записал изменяемые переменные $x_1, \dots, x_4$, в B6:E6 — коэффициенты цели $(3, 1, 4, 2)$. В B7 стоит $=\text{SUMPRODUCT}(B5:E5, B6:E6)$. Функция SUMPRODUCT (СУММПРОИЗВ) перемножает соответствующие элементы двух диапазонов и складывает произведения. Так вычисляется цель, а не ищется её минимум.

Строки B11:E13 содержат коэффициенты трёх ограничений. Формулы F11:F13 вычисляют левые части: $=\text{SUMPRODUCT}(\$B\$5:\$E\$5, B11:E11)$ и аналогично для строк 12 и 13. Знаки находятся в G11:G13, правые части 8, 5, 4 в H11:H13.

В Solver (Поиск решения) выбрал: цель B7; минимум; изменяемые ячейки B5:E5; $F11 \leq H11$, $F12 = H12$, $F13 \geq H13$; B5:E5 ≥ 0 ; метод «Симплекс LP». Запустил из нулевых значений. Начальные значения для Solver могут быть недопустимыми: допустимый план находит сам Solver. Получил $(0, 0, 0, 5)$ и 10. В таблице сравнения значение ручного решения равно 10, результат Excel связан формулой с B7.

Google Таблица содержит те же коэффициенты, формулы и найденный план. Настройки надстройки Excel Solver используются в исходном файле Excel; импорт в Google Таблицы сам по себе Solver не запускает.

7. Где что реализовано в программе

canonicalize Равенства, запас/избыток, нормализация отрицательных правых частей.

auxiliary_problem Начальный базис, искусственные столбцы, вспомогательная цель.

reduced_costs Оценки и значение цели.

pivot Отдельная функция пересчёта таблицы и замены базиса.

simplex Выбор столбца и строки, цикл шагов, оптимальность, неограниченность.

main_problem Вывод искусственных переменных, удаление их столбцов.

solve_lp Последовательность двух фаз, проверка допустимости, считывание ответа.

Код универсален для задач с неотрицательными переменными и знаками $\leq, =, \geq$. Коэффициенты своего варианта задаю отдельно. Для свободной по знаку переменной требуется предварительное преобразование. Готовых библиотек симплекс-метода нет: используется только `Fraction` из стандартной библиотеки для точных дробей. Номер итерации в программе начинается с нуля.

8. Построчный разбор кода

Нумерация ниже относится к коду четырёх ячеек, соединённых в один листинг, без добавленных комментариев. Каждая запись показывает саму строку и объяснение. В ноутбуке эти же пояснения записаны короткими комментариями.

Строка 1. Точные дроби без округления.

```
from fractions import Fraction as F
```

Строка 3. Приведение ограничений к равенствам.

```
def canonicalize(A, b, senses, c, direction='min'):
```

Аргументы отделяют алгоритм от конкретного варианта. По умолчанию решается минимум.

Строка 4. Копируем матрицу и переводим числа в дроби.

```
A = [[F(v) for v in row] for row in A]
```

Строка 5. Правые части в дробях; копия знаков.

```
b = list(map(F, b)); senses = list(senses)
```

Строка 6. Число исходных переменных и их имена.

```
n = len(c); names = [f'x{i+1}' for i in range(n)]
```

Строка 7. Перебираем ограничения.

```
for i in range(len(b)):
```

Строка 8. Проверяем знак правой части.

```
if b[i] < 0:
```

Строка 9. Умножаем всю строку на минус один.

```
A[i] = [-v for v in A[i]]; b[i] *= -1
```

Строка 10. Меняем направление неравенства.

```
senses[i] = {'<=': '>=', '>=': '<=', '=': '='}[senses[i]]
```

Строка 11. Добавляем переменные к неравенствам.

```
for i,s in enumerate(senses):
```

Строка 12. Проверяем допустимость знака.

```
if s not in ('<=', '>=', '='): raise ValueError('Unknown relation')
```

Строка 13. Равенству дополнительный столбец не нужен.

```
if s != '=':
```

Строка 14. Запас: +1; избыток: -1; в остальных строках 0.

```
for j,row in enumerate(A): row.append(F((1 if s=='<=' else -1) if i==j else  
↪ 0))
```

Строка 15. Имя дополнительной переменной по номеру строки.

```
names.append(f's{i+1}')
```

Строка 16. Цель для минимума; нули для дополнительных переменных.

```
cost = [F(v)*(1 if direction=='min' else -1) for v in c] + [F(0)]*(len(names)-n)
```

При максимизации меняются знаки исходной цели; у запаса и избытка цена нулевая.

Строка 17. Возвращаем каноническую задачу.

```
return A,b,cost,names
```

Строка 19. Строим начальный базис и вспомогательную цель.

```
def auxiliary_problem(A,b,names):
```

Строка 20. Размеры задачи, базис и строки без базиса.

```
m=len(b); n=len(names); basis=[]; missing=[]
```

Строка 21. Ищем базис для каждой строки.

```
for i in range(m):
```

Строка 22. Проверяем столбцы дополнительных переменных.

```
cols=[j for j,name in enumerate(names) if name.startswith('s') and
```

Поиск ограничен столбцами дополнительных переменных; искусственные добавляются только там, где такой единичный столбец отсутствует.

Строка 23. Единица в своей строке, нули в остальных.

```
all(A[k][j]==int(k==i) for k in range(m))]
```

Логическое выражение превращается в 1 или 0; all требует выполнения проверки во всех строках.

Строка 24. Берём найденный единичный столбец.

```
if cols: basis.append(cols[0])
```

Строка 25. Иначе назначаем искусственную переменную.

```
else: basis.append(n+len(missing)); missing.append(i)
```

Искусственные столбцы будут добавлены после n обычных; `missing` хранит номера строк, которым они нужны.

Строка 26. Добавляем искусственные столбцы и правые части.

```
T=[row+[F(int(i==k)) for k in missing]+[b[i]] for i,row in enumerate(A)]
```

Строка 27. Цель: сумма искусственных переменных.

```
return T,basis,[F(0)]*n+[F(1)]*len(missing),names+[f'a{k+1}' for k in missing]
```

Строка 29. Оценки и текущее значение цели.

```
def reduced_costs(T,basis,cost):
```

Строка 30. Оценка: коэффициент цели минус вклад базиса.

```
r=[cost[j]-sum(cost[basis[i]]*T[i][j] for i in range(len(T))) for j in  
→ range(len(cost))]
```

Формула $r_j = c_j - \sum_i c_{B_i} t_{ij}$; отрицательная оценка допускает улучшение минимума.

Строка 31. Цель: сумма коэффициентов базиса на правые части.

```
z=sum(cost[basis[i]]*T[i][-1] for i in range(len(T)))
```

Строка 32. Возвращаем оценки и цель.

```
return r,z
```

Строка 34. Пересчёт по разрешающему элементу.

```
def pivot(T,basis,row,col):
```

Строка 35. Нормируем разрешающую строку.

```
p=T[row][col]; T[row]=[v/p for v in T[row]]
```

Разрешающий элемент сохраняется до изменения строки. Все элементы, включая правую часть, делятся на него.

Строка 36. Перебираем строки таблицы.

```
for i in range(len(T)):
```

Строка 37. Разрешающую строку уже обработали.

```
if i!=row:
```

Строка 38. Обнуляем входящий столбец в остальных строках.

```
k=T[i][col]; T[i]=[v-k*w for v,w in zip(T[i],T[row])]
```

zip образует пары элементов текущей и разрешающей строк. k сохранён до изменения текущей строки.

Строка 39. Меняем базисную переменную.

```
basis[row]=col
```

Строка 41. Повторяем шаги до оптимальности.

```
def simplex(T,basis,cost,names,phase,history):
```

Строка 42. Защита от слишком долгого выполнения.

```
for step in range(1000):
```

Строка 43. Считаем оценки текущей таблицы.

```
r,z=reduced_costs(T,basis,cost)
```

Строка 44. Сохраняем фазу, шаг и имена базиса.

```
entry={'phase':phase,'step':step,'names':names[:],'basis':[names[i] for i in  
↪ basis],
```

Срез names[:] создаёт копию. Значения базиса переводятся из индексов в понятные имена.

Строка 45. Снимок таблицы, оценок и цели.

```
'T':[[str(v) for v in row] for row in T],'r':[str(v) for v in  
↪ r],'z':str(z)}
```

Строковые значения сохраняют точные дроби и не меняются при последующих пересчётах.

Строка 46. Добавляем таблицу в историю.

```
history.append(entry)
```

В историю добавлен объект entry; последующее добавление enter/leave относится к той же записи.

Строка 47. Столбцы, позволяющие уменьшить цель.

```
cols=[j for j,v in enumerate(r) if v<0]
```

Строка 48. Нет отрицательных оценок: оптимум.

```
if not cols: return
```

Строка 49. Первый подходящий столбец по индексу.

```
col=min(cols)
```

Выбор минимального индекса среди отрицательных оценок; это не выбор наиболее отрицательной оценки.

Строка 50. Берём только положительные элементы столбца.

```
rows=[i for i in range(len(T)) if T[i][col]>0]
```

Строка 51. Нет ограничивающей строки: цель не ограничена.

```
if not rows: raise ValueError('Objective unbounded')
```

Строка 52. Минимальное отношение; при равенстве индекс базиса.

```
row=min(rows,key=lambda i:(T[i][-1]/T[i][col],basis[i]))
```

Ключ является парой: сначала отношение, при равенстве индекс выходящей базисной переменной. Это правило Бланда.

Строка 53. Запоминаем входящую и выходящую переменные.

```
entry['enter']=names[col]; entry['leave']=names[basis[row]]
```

Строка 54. Сохраняем отношения для допустимых строк.

```
entry['ratios']=[str(T[i][-1]/T[i][col]) if T[i][col]>0 else '-' for i in  
↪ range(len(T))]
```

Отношение может быть нулевым; положительным должен быть коэффициент входящего столбца.

Строка 55. Выполняем разрешающий шаг.

```
pivot(T,basis,row,col)
```

Строка 56. Сообщаем о превышении числа шагов.

```
raise RuntimeError('Iteration limit')
```

Строка 58. Подготовка таблицы к основной задаче.

```
def main_problem(T,basis,n):
```

Строка 60. Идём с конца, чтобы безопасно удалять строки.

```
for i in range(len(T)-1,-1,-1):
```

При удалении строки обход с конца сохраняет номера ещё не обработанных строк.

Строка 61. Искусственная переменная ещё в базисе?.

```
if basis[i]>=n:
```

Строка 62. Ненулевая искусственная переменная: несовместность.

```
if T[i][-1]!=0: raise ValueError('Infeasible')
```

Строка 63. Ищем обычную переменную для замены.

```
cols=[j for j in range(n) if j not in basis and T[i][j]!=0]
```

После нулевой первой фазы можно заменить нулевую искусственную переменную по любому ненулевому подходящему элементу.

Строка 64. Выводим искусственную переменную из базиса.

```
if cols: pivot(T,basis,i,min(cols))
```

Строка 65. Удаляем избыточную строку $0 = 0$.

```
else: T.pop(i); basis.pop(i)
```

Если все обычные коэффициенты нулевые и правая часть ноль, строка является избыточной.

Строка 66. Удаляем искусственные столбцы, сохраняем b .

```
return [row[:n]+[row[-1]] for row in T],basis
```

Срез $row[:n]$ оставляет обычные столбцы, а $row[-1]$ возвращает правую часть, которая стояла после искусственных столбцов.

Строка 68. Объединяем все этапы решения.

```
def solve_lp(A,b,senses,c,direction='min'):
```

Строка 69. Получаем канонический вид.

```
C,rhs,cost,names=canonicalize(A,b,senses,c,direction)
```

Строка 70. Строим вспомогательную задачу.

```
T,basis,aux_cost,aux_names=auxiliary_problem(C,rhs,names)
```

Строка 71. Запускаем первую фазу и сохраняем историю.

```
history=[]; simplex(T,basis,aux_cost,aux_names,'I',history)
```

Строка 72. Вспомогательная цель должна стать нулём.

```
if reduced_costs(T,basis,aux_cost)[1]!=0: raise ValueError('Infeasible')
```

Второй результат `reduced_costs` — значение вспомогательной цели. Если оно положительно, допустимого плана исходной задачи нет.

Строка 73. Удаляем искусственные переменные.

```
T,basis=main_problem(T,basis,len(names))
```

Строка 74. Решаем исходную задачу во второй фазе.

```
simplex(T,basis,cost,names,'II',history)
```

Строка 75. Небазисные переменные равны нулю.

```
x=[F(0)]*len(names)
```

Строка 76. Базисные значения берём из правого столбца.

```
for i,j in enumerate(basis): x[j]=T[i][-1]
```

Строка 77. Возвращаем исходные x , исходную цель и таблицы.

```
return x[:len(c)],sum(F(v)*u for v,u in zip(c,x)),history
```

Срез $x[:len(c)]$ исключает дополнительные переменные. $zip(c,x)$ заканчивается на исходных коэффициентах; цель вычисляется с исходными знаками даже для максимизации.

Строка 79. Коэффициенты трёх ограничений варианта 4.

```
A = [[2, 1, 1, 0], [1, 0, 1, 1], [0, 1, 0, 1]]
```

Строки соответствуют трём ограничениям; столбцы — x_1, x_2, x_3, x_4 .

Строка 80. Правые части ограничений.

```
b = [8, 5, 4]
```

Строка 81. Знаки: неравенство, равенство, неравенство.

```
senses = ['<=', '=', '>=']
```

Строка 82. Коэффициенты исходной цели.

```
c = [3, 1, 4, 2]
```

Строка 83. Запускаем собственный симплекс-метод.

```
x, z, history = solve_lp(A, b, senses, c)
```

Строка 85. Печатаем сохранённые таблицы.

```
for t in history:
```

Строка 86. Номер фазы и шага.

```
print('Фаза', t['phase'], 'шаг', t['step'])
```

Строка 87. Заголовки столбцов.

```
print('Базис | ' + ' | '.join(t['names']) + ' | b')
```

Строка 88. Базисная переменная и её строка.

```
for name, row in zip(t['basis'], t['T']):
```

Каждое имя базисной переменной связывается со своей строкой таблицы.

Строка 89. Печатаем коэффициенты строки.

```
print(name + ' | ' + ' | '.join(row))
```

Строка 90. Печатаем оценки и цель.

```
print('r | ' + ' | '.join(t['r']), 'Цель:', t['z'])
```

Строка 91. Был ли переход из этой таблицы?.

```
if 'enter' in t:
```

Строка 92. Показываем смену базиса.

```
print('Входит:', t['enter'], 'Выходит:', t['leave'])
```

Строка 93. Пустая строка между таблицами.

```
print()
```

Строка 94. Оптимальная точка.

```
print('x* =', tuple(map(str, x)))
```

Fraction переводится в строку для читаемого вывода; tuple оформляет план как кортеж.

Строка 95. Оптимальное значение цели.

```
print('z =', z)
```

9. Вопросы преподавателя и короткие ответы

Почему две искусственные переменные? Во втором ограничении равенство, в третьем знак $\geq$ и столбец избытка с коэффициентом -1 . Для начального единичного базиса добавлены a_2, a_3 .

Почему начальный план $(8, 5, 4)$? Небазисные переменные обнулены; из равенств получаются $s_1 = 8, a_2 = 5, a_3 = 4$. Это значения базисных переменных вспомогательной задачи.

Почему вспомогательная цель равна девяти? $w = a_2 + a_3 = 5 + 4 = 9$; запас s_1 в эту цель не входит.

Почему сначала вводится x_1 , хотя оценка x_4 ниже? Использую правило выбора первого отрицательного столбца по индексу. Оно допустимо и совпадает в ручном решении и коде.

Зачем делить правые части? Минимальное отношение задаёт максимальное допустимое увеличение входящей переменной без отрицательных базисных значений.

Почему не берём нулевой или отрицательный элемент столбца? Он не задаёт верхней границы увеличения входящей переменной в этой строке.

Чем базисные переменные отличаются от небазисных? Небазисные принимаются равными нулю. Базисные определяются системой; в текущей таблице их значения находятся в правом столбце.

Когда задача несовместна? Если минимум суммы искусственных переменных положителен. При нулевой сумме все неотрицательные искусственные переменные обнулились.

Когда задача неограниченна? Есть улучшающая оценка, но во входящем столбце нет положительных элементов; выходящую строку выбрать нельзя.

Почему оценки базисных переменных нулевые? Их столбцы единичные, поэтому в формуле оценки вычитается их собственный коэффициент цели.

Почему остановились? В последней таблице все оценки неотрицательны. При этой записи минимизации допустимое увеличение небазисных переменных не уменьшает цель.

Почему $s_1 = 8$, $s_3 = 1$? Первое ограничение использует ноль из восьми; третье имеет левую часть пять при требуемых четырёх.

Выполнены рекомендации к коду? Да: входные коэффициенты отделены от алгоритма; пересчёт в pivot; этапы выделены в функции; готовый симплекс-решатель не используется.

Есть ли отдельная функция всего шага? pivot выполняет пересчёт и смену базиса. Выбор строки и столбца расположен в simplex, который вызывает pivot.

Что делает SUMPRODUCT, а что Solver? SUMPRODUCT вычисляет цель или левую часть. Solver изменяет переменные, чтобы найти минимум при ограничениях.

Нужен ли общий отчёт? По заданию нужен PDF с Ф.И.О., вариантом, ходом решения, ответом и рефлексивным выводом. Ручной PDF выполняет эту роль. Поле «Поток» было убрано по моей просьбе; в исходном задании оно указано. Дополнительный сводный отчёт отдельно не требуется.

Почему можно проверить ответ без таблиц? Равенство даёт $z = 10 + x_1 + x_2 + 2x_3 \geq 10$. Найденная допустимая точка достигает этой границы. Это независимая проверка; требуемый симплекс-метод выполнен отдельно.